Practice Set 1 — Final Results and Answers

Friday 23 Oct 2026 · Score: **26 / 30 (87%)** · AWS pass mark is 750/1000; aim for 75%+ on practice sets

Verdict
26/30 = 87% on the topics covered so far — a strong first result. All four misses are 'which option is the precise, intended mechanism' errors (exact API, exact limit, exact service support), which is exactly what the exam tests. They are now in your mistakes log below.

1. Score by exam task

Task	Score	Status
1.3	1 / 1	✓ strong
1.4	4 / 5	½ review
1.5	2 / 3	½ review
2.1	14 / 16	½ review
2.4	1 / 1	✓ strong
3.1	1 / 1	✓ strong
3.2	1 / 1	✓ strong
4.3	1 / 1	✓ strong
5.1	1 / 1	✓ strong

2. Mistakes log — the four misses

Q	Task	Your answer	Correct
2	1.4	B	D
7	2.1	C	B
17	1.5	A	B
29	2.1	A	B

Rule to remember	
Q2 — vector stores	**DynamoDB is not a Knowledge Base vector store.** Supported: OpenSearch Serverless, Aurora PostgreSQL (pgvector), Neptune Analytics, S3 Vectors, and third parties (Pinecone, MongoDB Atlas, Redis). 'Already runs Aurora + SQL joins + no new service' → **Aurora pgvector**.
Q7 — tool results	Tool outputs go in **toolResult blocks in ONE user message** (matching toolUseIds), never in the system prompt.
Q17 — Retrieve vs RetrieveAndGenerate vs agents	Need **custom logic between retrieval and generation** (filter chunks, add other data) → **Retrieve + Converse**. InvokeAgent adds an unnecessary agent loop.
Q29 — Step Functions timeouts	Escalate when nobody answers → **TimeoutSeconds / HeartbeatSeconds on the callback task + Catch States.Timeout**. **Express max = 5 minutes**; Standard up to 1 year.

3. Why the misses happened — reading traps

Two of the four misses (Q7 and Q29) were **concept-right, letter-wrong**: the chosen option started with the correct idea and ended with the wrong mechanism. Q10 (Standard workflow) was answered correctly, then Q29 picked an Express workflow — a sign of fatigue, not a knowledge gap.

Q	Trap	Habit to build
7	Option C began 'Run both tools…' (correct) and ended '…in the system prompt' (wrong).	Read every option to the **last word**; compare options that share an opening.
29	Express workflow chosen despite knowing its 5-minute limit (Q10 was right).	Re-check hard limits (time, size, quotas) against the scenario's numbers (48 hours).
2	DynamoDB assumed to be a vector store.	Memorise the supported KB vector stores list.
17	Agent chosen where a simple Retrieve + Converse pipeline fits.	'Custom logic between retrieval and generation' → Retrieve, then build the prompt.

Exam technique
Use **Flag for review** on any question where two options start the same way, and revisit flagged questions at the end. Concept score on this set, excluding the two reading errors: **28/30**.

4. Clarification — Q7 and the agent loop

In the Converse API a **'user message'** means a message with **role: "user"** in the messages array — not a message to or from the human. The API only has two roles and they must alternate, so tool results (which come from your side) are sent in a role "user" message.

```
Call 1 -> messages = [
  { role: "user",      content: [ text: "Status of 1002 and 1003?" ] }       <- the human's question
]
<- model: stopReason = tool_use

Call 2 -> messages = [
  { role: "user",      content: [ text: "Status of 1002 and 1003?" ] },
  { role: "assistant", content: [ toolUse(id=a, get_order_status 1002),      <- model reply, appended as-is
                                  toolUse(id=b, get_order_status 1003) ] },
  { role: "user",      content: [ toolResult(id=a, "PROCESSING"),            <- YOUR tool results:
                                  toolResult(id=b, "DELIVERED") ] }             role "user", ONE message
]
<- model: stopReason = end_turn -> only NOW show the answer to the human
```

- Until stopReason is not **tool_use**, nothing goes to the human — keep looping.
- Every call re-sends the **whole history**: question, every assistant message, every tool-result message.

	Who runs the loop	Who runs the tool	How results return
Converse API, your own loop	**You**	You	toolResult blocks in a role 'user' message
Bedrock Agents, return of control	**Bedrock Agents**	Your app (that one action)	Next InvokeAgent call, sessionState.returnControlInvocationResults
Bedrock Agents, Lambda action group	Bedrock Agents	Lambda	Handled by Bedrock

5. Readiness on the topics covered so far

Task	Confidence	Evidence and gap
2.1 Agents and tools	High (~85%)	14/16; agent loop well understood; misses were reading errors.
1.4 Vector stores	Medium-high (~75%)	Chunking, sync, GraphRAG solid. Gap: supported vector stores.
1.5 Retrieval	Medium-high (~75%)	Hybrid, rerank, filters solid. Gap: Retrieve vs RetrieveAndGenerate.

Overall: about **8 out of 10** on real exam questions for these tasks. Expect real-exam scores 5–10 points below self-made practice sets. To raise it: 10 minutes of mistakes-log review each morning, read every option to the end, and Practice Set 2 (31 Oct) will re-test 5 questions from Days 1–3.

6. Every question explained

Question 1 · Task 1.5 · ✓ you answered C, E, correct C, E

(Choose TWO) A law firm uses an Amazon Bedrock Knowledge Base over 20,000 contracts. Natural-language questions work well, but queries containing exact clause references such as 'Section 14.3(b)' often return irrelevant chunks, and when the right chunk is returned it is usually ranked 7th or 8th. The firm wants to fix both problems with minimal custom development. Which actions should it take?

A. Re-create the data source with semantic chunking.
✗ — Chunking changes how text is split, not how exact tokens are matched or ranked.

B. Increase the number of retrieved results to 100.
✗ — More results add noise and tokens; it doesn't put the best chunk first or fix exact-match misses.

C. Switch the Knowledge Base search type to hybrid search. (your choice)
✓ **CORRECT** — Hybrid search adds keyword (BM25) matching, which catches exact tokens such as 'Section 14.3(b)'.

D. Fine-tune the embedding model on legal terminology.
✗ — Heavy effort (custom training) and still weak on exact strings — not 'minimal development'.

E. Add an Amazon Bedrock rerank model to the retrieval configuration. (your choice)
✓ **CORRECT** — A reranker re-scores retrieved chunks so the most relevant one moves to the top.

Question 2 · Task 1.4 · ✗ you answered B, correct D

A company is building a RAG application on Amazon Bedrock Knowledge Bases. Its data team already operates Amazon Aurora PostgreSQL and wants to store the vectors where it can also join them with existing relational metadata using SQL. The company wants to avoid introducing and operating a new database service. Which vector store should it choose?

A. Amazon OpenSearch Serverless
✗ — Would introduce and operate a new service — the scenario says avoid that.

B. Amazon DynamoDB (your choice)
✗ — DynamoDB is not a supported Bedrock Knowledge Base vector store.

C. Amazon Neptune Analytics
✗ — Neptune Analytics is for GraphRAG (relationships), and is a new service for this team.

D. Amazon Aurora PostgreSQL with the pgvector extension
✓ **CORRECT** — Aurora PostgreSQL + pgvector is a supported KB vector store and allows SQL joins with existing relational data.

Question 3 · Task 1.5 · ✓ you answered C, correct C

A SaaS provider serves 400 tenants from a single Bedrock Knowledge Base. Documents from one tenant must never appear in another tenant's answers. The solution must enforce isolation reliably with the least operational overhead. What should a developer do?

A. Create one Amazon Bedrock guardrail per tenant with denied topics for the other tenants' names.
✗ — Guardrails filter content categories; 400 guardrails with name lists is fragile and high overhead.

B. Post-process every response with an AWS Lambda function that removes other tenants' names.
✗ — Post-processing after generation is too late and unreliable; the data already reached the model.

C. Attach a tenant_id metadata attribute to every document and apply a metadata filter in each Retrieve call, set from the caller's authenticated tenant. (your choice)
✓ **CORRECT** — Metadata filtering at retrieval time ensures other tenants' chunks never reach the prompt. Reliable and low overhead.

D. Add a system prompt instruction telling the model to answer only with the current tenant's documents.

✗ — Prompts are not an access-control mechanism — the model can still be shown other tenants' chunks.

Question 4 · Task 1.3 · ✓ you answered B, correct B

An insurer ingests scanned claim forms as PDFs. Each contains tables of line items and some handwritten notes. The Knowledge Base returns wrong totals because the table structure is lost during ingestion. Which change most directly fixes the problem?

A. Switch the search type from semantic to hybrid.
✗ — Search type affects matching, not how the document was parsed.

B. Configure the data source to use foundation-model parsing or Amazon Bedrock Data Automation as the parser. (your choice)
✓ **CORRECT** — FM parsing / Bedrock Data Automation understands tables, layouts and handwriting during ingestion.

C. Increase the fixed chunk size to 1,000 tokens.
✗ — Bigger chunks don't restore table structure lost during parsing.

D. Use a higher-dimension embedding model.
✗ — Embedding dimensions don't fix structure that was lost before embedding.

Question 5 · Task 1.4 · ✓ you answered C, correct C

Product manuals in an Amazon S3 bucket are updated several times a day. The Bedrock Knowledge Base must reflect changes within one hour, using a serverless approach with the least effort. What should the developer implement?

A. Delete and re-create the Knowledge Base every hour.
✗ — Re-creating the KB is slow, wasteful and causes downtime.

B. Enable S3 Versioning on the bucket so the Knowledge Base detects new object versions automatically.
✗ — S3 Versioning does not trigger Knowledge Base syncs.

C. An Amazon EventBridge Scheduler schedule that calls StartIngestionJob for the data source every hour. (your choice)
✓ **CORRECT** — Scheduled StartIngestionJob is serverless and syncs incrementally (only changed documents).

D. A Lambda function that re-embeds every document and writes vectors directly into the OpenSearch index.
✗ — Rebuilds what the KB already manages; high effort and bypasses the managed pipeline.

Question 6 · Task 3.1 · ✓ you answered B, correct B

A customer support assistant built on a Bedrock Knowledge Base sometimes states refund amounts that do not appear anywhere in the retrieved policy documents. The company must block these unsupported responses with minimal custom code. What should a developer do?

A. Switch to a larger foundation model.
✗ — A larger model can still hallucinate; it doesn't enforce grounding.

B. Configure an Amazon Bedrock guardrail with a contextual grounding check and apply it to responses. (your choice)
✓ **CORRECT** — The contextual grounding check blocks responses not supported by the retrieved source text.

C. Set the model's temperature to 0.
✗ — Temperature 0 makes output more deterministic, not grounded in the sources.

D. Add metadata filters to the retrieval query.
✗ — Metadata filters control which documents are retrieved, not whether the answer is faithful to them.

Question 7 · Task 2.1 · ✗ you answered C, correct B

A developer builds a custom agent loop with the Amazon Bedrock Converse API. A response returns stopReason tool_use with two toolUse blocks. What should the application do next?

A. Run only the first tool and ignore the second until the next turn.
✗ — Calling again without results gives the model nothing new — it will ask for the tools again.

B. Run both tools, append the assistant message, then send one user message containing two toolResult blocks with the matching toolUseId values.
✓ **CORRECT** — Correct: append the assistant message, then ONE user message containing all toolResult blocks with matching toolUseIds.

C. Run both tools and place their outputs in the system prompt of the next call. (your choice)
✗ — Tool results belong in toolResult blocks in a user message, not in the system prompt.

D. Call Converse again with the same messages so the model can confirm which tool to use.
✗ — The model expects a result for every toolUse; skipping one breaks the conversation.

Question 8 · Task 2.1 · ✓ you answered B, correct B

An enterprise wants agents to use 300 internal REST APIs as tools. In testing, the agent often picks the wrong API, and prompts become very large because every tool definition is sent on every call. Which solution addresses both problems with the least custom code?

A. Increase the model's maximum output tokens.
✗ — Output tokens don't affect tool choice or the size of the input prompt.

B. Expose the APIs through Amazon Bedrock AgentCore Gateway with OpenAPI targets and semantic tool search enabled, so agents retrieve only the relevant tools. (your choice)
✓ **CORRECT** — Gateway exposes the APIs as MCP tools; semantic tool search sends only the relevant tools, fixing both problems.

C. Create one dedicated agent for each API.
✗ — 300 agents is huge operational overhead and doesn't solve tool selection.

D. Fine-tune the foundation model on the API documentation.
✗ — Fine-tuning is costly, slow and doesn't shrink the tool definitions sent each call.

Question 9 · Task 2.1 · ✓ you answered B, E, correct B, E

(Choose TWO) A team operates an existing Amazon Bedrock agent (Agents Classic). The agent must stop guessing missing parameters and ask the user for them instead. It must also obtain the user's explicit approval before running the delete_account function. Which configurations meet these requirements?

A. Raise the model's temperature so it explores alternatives.
✗ — Higher temperature increases randomness — the opposite of what's needed.

B. Enable user confirmation (requireConfirmation) on the delete_account function. (your choice)
✓ **CORRECT** — requireConfirmation makes the agent ask for explicit approval before running the function.

C. Attach a Knowledge Base containing the account policies.
✗ — A KB provides knowledge; it neither asks the user nor gates destructive actions.

D. Enable the Code Interpreter action group.
✗ — Code Interpreter runs code; unrelated to asking for parameters or approvals.

E. Enable the built-in User input action group. (your choice)
✓ **CORRECT** — User input lets the agent ask the user for missing information instead of guessing.

Question 10 · Task 2.1 · ✓ you answered C, correct C

A bank's mortgage process has fixed steps: extract documents, generate a risk summary with an LLM, wait for a human underwriter's approval (which can take up to 3 days), then notify the applicant. The bank needs a full audit history and automatic retries. What should a developer use?

A. An Amazon Bedrock agent with long-term memory to track each application.
✗ — The steps are fixed — a workflow is cheaper, deterministic and auditable; an agent adds unpredictability.

B. A chain of Lambda functions that polls a database for the approval every minute.
✗ — Polling Lambdas cost money, add code and give no built-in audit trail.

C. An AWS Step Functions Standard workflow with a Bedrock InvokeModel task and a .waitForTaskToken callback step for the approval. (your choice)
✓ **CORRECT** — Standard workflows run up to 1 year with full execution history; waitForTaskToken pauses for the approval; retries are built in.

D. An AWS Step Functions Express workflow with a callback step for the approval.
✗ — Express workflows run at most 5 minutes — impossible for a 3-day approval.

Question 11 · Task 2.1 · ✓ you answered A, correct A

A company has agents built with LangGraph and with Strands Agents. It needs serverless hosting with per-user session isolation, support for sessions that run for several hours, and no container clusters to manage. Which service fits best?

A. Amazon Bedrock AgentCore Runtime (your choice)
✓ **CORRECT** — AgentCore Runtime: serverless, framework-agnostic, per-session microVM isolation, long-running sessions.

B. Amazon Bedrock Agents (Classic) inline agents
✗ — Agents Classic doesn't host LangGraph/Strands code, and new accounts can't use it.

C. AWS Lambda, one function per agent
✗ — Lambda has a 15-minute limit and no built-in session isolation for agents.

D. Amazon ECS on AWS Fargate behind an Application Load Balancer
✗ — Works, but you manage containers, scaling and session isolation yourself.

Question 12 · Task 2.1 · ✓ you answered C, correct C

An internal agent must create issues in each user's own GitHub repositories using that user's permissions. Security policy forbids storing tokens in agent code or environment variables. What should the developer use?

A. An Amazon Cognito identity pool that maps users to GitHub accounts.
✗ — Cognito identity pools hand out AWS credentials, not GitHub OAuth tokens.

B. A single shared GitHub personal access token stored in a Lambda environment variable.
✗ — A shared token acts as one identity (not each user) and breaks the 'no tokens in env vars' rule.

C. An Amazon Bedrock AgentCore Identity outbound OAuth credential provider that obtains user-delegated tokens from its token vault for each authenticated user. (your choice)
✓ **CORRECT** — AgentCore Identity outbound OAuth gets user-delegated tokens from a token vault — each user's own permissions.

D. An IAM role with a trust policy for GitHub.
✗ — IAM roles authorize access to AWS resources, not GitHub.

Question 13 · Task 2.1 · ✓ you answered D, correct D

A travel assistant must remember each user's preferences (aisle seat, vegetarian meals) across conversations months apart. Raw conversation history must expire after 30 days for compliance. What should the developer implement?

A. Store full transcripts in Amazon DynamoDB and include the entire history in every prompt.
✗ — Huge prompts and cost, and it breaks the 30-day retention requirement.

B. Use a model with a larger context window.
✗ — A bigger context window doesn't persist anything across sessions.

C. Ingest chat logs nightly into a Bedrock Knowledge Base.
✗ — A KB is shared document search, not per-user memory, and would keep raw logs.

D. Amazon Bedrock AgentCore Memory with a 30-day short-term event expiry and a long-term user-preference strategy using per-user namespaces. (your choice)
✓ **CORRECT** — Short-term events expire at 30 days; the user-preference strategy keeps extracted preferences long-term, per user.

Question 14 · Task 2.1 · ✓ you answered B, correct B

A customer service platform has billing, technical-support and sales agents. Analysis shows 95% of requests need exactly one specialist. The company wants the lowest latency and cost. Which design should it use?

A. A supervisor agent that sends every request to all three specialists and merges the answers.
✗ — Calling all three specialists for every request triples cost and latency.

B. A router/classifier that sends each request to the single best specialist, such as AWS Agent Squad or a supervisor in routing mode. (your choice)
✓ **CORRECT** — A router sends each request to exactly one specialist — the cheapest, fastest design for single-specialist traffic.

C. One agent that holds all 120 tools from the three teams.
✗ — 120 tools in one agent confuses tool selection and bloats every prompt.

D. A swarm in which specialists pass the request among themselves.
✗ — Swarms are unpredictable and add hand-off calls.

Question 15 · Task 2.1 · ✓ you answered C, correct C

Five teams build agents with different frameworks (Strands, LangGraph, CrewAI), and each team has written its own integration to the same internal ticketing and CRM tools. The company wants to write each tool once, reuse it across all frameworks and manage its authentication centrally. What should it do?

A. Rebuild every agent as an Amazon Bedrock Flow.
✗ — Flows are fixed workflows; rebuilding agents as flows loses agent behaviour.

B. Paste each tool's OpenAPI specification into every agent's system prompt.
✗ — Pasting specs into prompts wastes tokens and doesn't execute or secure anything.

C. Expose the tools as MCP servers served centrally, for example through Amazon Bedrock AgentCore Gateway. (your choice)
✓ **CORRECT** — MCP is the standard tool protocol every framework supports; Gateway centralizes hosting and auth.

D. Publish a shared Python library that every team imports.
✗ — A Python library doesn't help non-Python frameworks or centralize auth.

Question 16 · Task 1.4 · ✓ you answered A, correct A

A manufacturer's maintenance manuals contain long procedures. With small fixed-size chunks, answers are precise but miss the preceding safety steps; with large chunks, retrieval returns noisy, loosely related text. The team wants precise matching and full surrounding context in the prompt. Which chunking strategy should it use?

A. Hierarchical chunking with small child chunks and larger parent chunks (your choice)
✓ **CORRECT** — Hierarchical: small child chunks match precisely; the larger parent chunk gives the

model full context.

B. Semantic chunking with a very high breakpoint threshold
✗ — Semantic chunking splits by topic shifts; it doesn't give 'precise match + bigger context'.

C. Fixed-size chunking with 1,500-token chunks and 50% overlap
✗ — Huge chunks are noisy and the scenario already says large chunks fail.

D. No chunking, so each manual is one chunk
✗ — Whole manuals as one chunk — extremely noisy and may exceed limits.

Question 17 · Task 1.5 · ✗ you answered A, correct B

A developer must retrieve policy chunks from a Bedrock Knowledge Base, drop chunks that fail custom business rules, add the customer's account details from Amazon DynamoDB to the prompt, and then call a foundation model. Which approach should the developer use?

A. Call InvokeAgent with the Knowledge Base attached. (your choice)
✗ — StartIngestionJob syncs data; it doesn't answer queries.

B. Call the Retrieve API, apply the business rules in application code, build the prompt, then call the Converse API.
✓ **CORRECT** — Retrieve returns chunks only, so you can filter them, add DynamoDB data and build your own prompt for Converse.

C. Call StartIngestionJob with a custom Lambda transformation.
✗ — RetrieveAndGenerate is one managed call — no hook to apply business rules or add DynamoDB data between retrieval and generation.

D. Call RetrieveAndGenerate with a custom prompt template.
✗ — Agents (Classic) add an orchestration loop you don't need, and new accounts can't create them.

Question 18 · Task 1.4 · ✓ you answered D, correct D

A compliance team needs answers that depend on relationships spread across thousands of documents, such as 'Which of our suppliers are subsidiaries of companies headquartered in a sanctioned country?'. Standard vector retrieval returns isolated chunks that miss these connections. Which solution is most appropriate?

A. Add a rerank model to the retrieval configuration.
✗ — Reranking reorders chunks; it doesn't connect entities across them.

B. Switch the existing OpenSearch Serverless store to hybrid search.
✗ — Hybrid search improves term matching, not multi-hop relationships.

C. Increase the chunk size so more relationships fit in each chunk.
✗ — Bigger chunks can't capture relationships spread over thousands of documents.

D. Use a Bedrock Knowledge Base with Amazon Neptune Analytics as the vector store to enable GraphRAG. (your choice)
✓ **CORRECT** — GraphRAG with Neptune Analytics connects entities across documents for relationship questions.

Question 19 · Task 5.1 · ✓ you answered D, correct D

A team changed the chunking strategy of its Knowledge Base and must prove retrieval quality improved before rollout. It wants a managed approach that scores retrieval with LLM-as-a-judge metrics such as context relevance and context coverage on a prompt dataset. What should it use?

A. A Bedrock guardrail contextual grounding check
✗ — Grounding checks block unfaithful answers at runtime; they don't evaluate retrieval quality.

B. Manual spot checks of 20 answers by the support team
✗ — Manual spot checks aren't managed, repeatable or statistically meaningful.

C. Amazon CloudWatch metrics for the Knowledge Base
✗ — CloudWatch shows operational metrics, not retrieval quality scores.

D. An Amazon Bedrock Evaluations RAG evaluation (retrieve-only) job run against both Knowledge Bases with the same prompt dataset (your choice)
✓ **CORRECT** — Bedrock Evaluations RAG (retrieve-only) jobs score context relevance/coverage with LLM-as-a-judge, comparing both KBs.

Question 20 · Task 1.4 · ✓ you answered A, correct A

Documents in a Knowledge Base were embedded with Amazon Titan Text Embeddings V2. A developer changes the application to embed user queries with a different embedding model before running a vector search, and retrieval quality collapses. What is the cause and the correct fix?

A. Queries and documents must be embedded with the same model; to switch models, create a new Knowledge Base with the new embedding model and re-ingest the documents. (your choice)
✓ **CORRECT** — Query and document vectors must come from the same model; a new embedding model means a new KB + re-ingestion.

B. The rerank model is missing; add one to fix ranking.
✗ — A reranker only reorders what retrieval returns; retrieval itself is broken.

C. Hybrid search is disabled; enable it to compensate.
✗ — Hybrid search can't compensate for a broken semantic half.

D. The vector index is too small; increase the number of results.
✗ — The number of results doesn't fix vectors from incompatible embedding spaces.

Question 21 · Task 2.4 · ✓ you answered C, correct C

In a Converse API tool loop, a tool call fails because a downstream inventory API times out. The model should be able to recover, for example by retrying differently or telling the user. What should the application send back?

A. Nothing; drop the toolUse block and call Converse again without a result.
✗ — Every toolUse needs a toolResult; dropping it breaks the conversation contract.

B. The same tool request again in an unbounded retry loop until it succeeds.
✗ — Unbounded retries can loop forever and burn tokens; use bounded retries with backoff.

C. A toolResult with the same toolUseId, status set to error, and an error message as content. (your choice)
✓ **CORRECT** — Return a toolResult with status 'error' and a message — the model can then retry differently or inform the user.

D. An exception message directly to the end user, ending the conversation.
✗ — Ending the conversation is poor UX when the model could recover.

Question 22 · Task 2.1 · ✓ you answered C, correct C

A company must classify 2 million documents per month into five fixed categories. The steps never change, results are not needed in real time, and cost is the top priority. Which design is most appropriate?

A. A swarm of agents that vote on each document's category
✗ — Voting multiplies calls and cost for a simple classification.

B. A supervisor agent that delegates each document to five category specialists
✗ — A supervisor multiplies model calls per document — far more expensive.

C. One foundation model call per document with a structured output format, run as a batch job (your choice)
✓ **CORRECT** — A single structured call per document, run as batch inference (~50% cheaper), fits fixed steps, no real-time need and cost focus.

D. An agent with classification tools that reasons about each document
✗ — Agents add reasoning loops and cost per document; nothing here needs runtime decisions.

Question 23 · Task 2.1 · ✓ you answered C, correct C

A developer updates an action group's OpenAPI schema on an existing Bedrock agent (Agents Classic). When testing through the built-in test alias, the agent still behaves as before. What should the developer do?

A. Wait up to 24 hours for the change to propagate.
✗ — Changes don't propagate by waiting; DRAFT must be prepared.

B. Delete and re-create the production alias.
✗ — Deleting the production alias affects production and doesn't apply DRAFT changes.

C. Prepare the agent (PrepareAgent) and test again against the test alias, which points to the DRAFT version. (your choice)
✓ **CORRECT** — PrepareAgent packages DRAFT changes; the built-in test alias (TSTALIASID) points to DRAFT.

D. Enable tracing; the change is applied but not visible.
✗ — Tracing shows behaviour; it doesn't apply unprepared changes.

Question 24 · Task 2.1 · ✓ you answered A, correct A

A retail agent must check stock in an on-premises inventory system that is reachable only from the company's own application servers. The security team prohibits any network path from AWS into the data center and does not want new network infrastructure. How should the action be implemented with Bedrock Agents (Classic)?

A. An action group configured with return of control, so the application server runs the inventory call and returns the result to the agent (your choice)
✓ **CORRECT** — Return of control hands the parameters to the application server, which already reaches the inventory system.

B. The Code Interpreter action group calling the inventory system
✗ — Code Interpreter runs in a sandbox and can't reach the on-premises system.

C. A Knowledge Base synced nightly from an inventory export
✗ — A nightly export isn't real-time stock data.

D. A Lambda function in a VPC connected to the data center through AWS Direct Connect
✗ — Direct Connect creates exactly the network path the security team forbids, and is new infrastructure.

Question 25 · Task 2.1 · ✓ you answered A, correct A

During a session, every action group call of a Bedrock agent (Agents Classic) needs the authenticated customer's ID. The agent must not ask the user for it, and the model must not be able to change it. How should the application provide the ID?

A. Pass it in sessionAttributes on InvokeAgent; the Lambda function reads it from the event's sessionAttributes. (your choice)
✓ **CORRECT** — sessionAttributes persist for the session and are passed to the Lambda event, outside the model's control.

B. Store it in the agent's long-term memory.
✗ — Long-term memory stores summaries across sessions — not a reliable per-call parameter.

C. Include the customer ID in the text of each user prompt.
✗ — Prompt text can be changed or ignored by the model and the user.

D. Pass it in promptSessionAttributes on the first turn only.
✗ — promptSessionAttributes last one turn and are inserted into the prompt, so they're visible to the

model.

Question 26 · Task 4.3 · ✓ you answered C, correct C

A production agent hosted on Amazon Bedrock AgentCore Runtime shows latency spikes. The team must find which tool call is slow and how many tokens each step uses. What should it use?

A. AWS CloudTrail management events
✗ — CloudTrail records API calls (who did what), not per-step latency or tokens.

B. Amazon S3 server access logs
✗ — S3 access logs are unrelated to agent execution.

C. AgentCore Observability traces in Amazon CloudWatch, with CloudWatch Transaction Search enabled (your choice)
✓ **CORRECT** — AgentCore Observability traces show each step, tool call, latency and tokens in CloudWatch (needs Transaction Search).

D. AWS Config rules
✗ — Config tracks resource configuration compliance, not runtime performance.

Question 27 · Task 3.2 · ✓ you answered A, C, correct A, C

(Choose TWO) A company exposes Lambda functions as tools through Amazon Bedrock AgentCore Gateway. Only authenticated agent applications may call the gateway, and the gateway must call the Lambda functions with least privilege. Which configurations meet these requirements?

A. Give the gateway's service role lambda:InvokeFunction scoped to the specific function ARNs. (your choice)
✓ **CORRECT** — Scoping lambda:InvokeFunction to the specific function ARNs is least privilege for the outbound side.

B. Make each Lambda function URL public so the gateway can reach it.
✗ — Public function URLs remove authentication — the opposite of the requirement.

C. Configure the gateway's inbound authorization with a JWT authorizer, such as an Amazon Cognito user pool. (your choice)
✓ **CORRECT** — Inbound JWT authorization (e.g. Cognito) ensures only authenticated callers reach the gateway.

D. Attach the AdministratorAccess policy to the gateway's service role.
✗ — AdministratorAccess violates least privilege.

E. Embed AWS access keys for the Lambda functions in the tool schema.
✗ — Embedding access keys in a schema leaks long-term credentials.

Question 28 · Task 2.1 · ✓ you answered D, correct D

An analytics agent must load a user-uploaded CSV file, run pandas code it writes itself and return a chart. The generated code must run in an isolated sandbox. Which capability should the agent use?

A. Amazon Bedrock AgentCore Gateway
✗ — Gateway exposes tools; it doesn't execute arbitrary generated code.

B. Amazon Bedrock AgentCore Browser
✗ — Browser drives websites, not code execution.

C. A Lambda function that runs the generated code with eval()
✗ — eval() of generated code in Lambda is unsafe and not isolated per session.

D. Amazon Bedrock AgentCore Code Interpreter (your choice)
✓ **CORRECT** — Code Interpreter runs agent-generated code (pandas, charts) in an isolated sandbox.

Question 29 · Task 2.1 · ✗ you answered A, correct B

A Step Functions workflow pauses at a .waitForTaskToken step for a manager's approval. If nobody responds within 48 hours, the request must be escalated automatically. How should the developer implement this?

A. Convert the workflow to an Express workflow. (your choice)
✗ — Express workflows run max 5 minutes — can't wait 48 hours at all.

B. Set TimeoutSeconds (or HeartbeatSeconds) on the callback task and add a Catch for States.Timeout that routes to an escalation branch.
✓ **CORRECT** — Timeout/HeartbeatSeconds on the callback task raises States.Timeout; a Catch routes it to escalation.

C. Add a Lambda function that polls the approval status every minute for 48 hours.
✗ — Polling adds cost and code for something Step Functions does natively.

D. Add a 48-hour Wait state before the callback task.
✗ — A Wait state just delays before the task; it can't time out the approval.

Question 30 · Task 2.1 · ✓ you answered D, correct D

A research assistant must produce one report that combines analysis from market-data, legal and financial specialist agents. The specialists work independently of each other. The company wants to minimize end-to-end latency. Which design is best?

A. One agent with all specialists' tools that works through them one by one
✗ — One agent working through everything sequentially is slow and mixes concerns.

B. A sequential workflow that calls the three specialists one after another
✗ — Sequential calls add each specialist's latency together.

C. A router that sends the request to the single most relevant specialist
✗ — A router picks one specialist, but the report needs all three.

D. A supervisor (agents-as-tools) that invokes the three specialists in parallel and synthesizes their results (your choice)
✓ **CORRECT** — Independent specialists run in parallel under a supervisor; latency ≈ the slowest one plus synthesis.